A playful, private space for everything you don’t want to forget

Expanded product specification • architecture • workflows • stack • AI-assisted development plan

Working title: intentionally undecided. This document does not lock in a product name.

A lightweight thought-capture companion that can grow into a personal context system: notes, images, links, a watchlist and permissioned AI access through MCP. The operator should not hold readable user notes; privacy is a core architectural constraint.

What this document adds

- End-to-end architecture and data-flow diagrams (described in text and component maps).
- Concrete technology choices, libraries, development tools and deployment boundaries.
- Workflows for capture, browser extension, MCP, privacy, backup and the later watchlist module.
- A practical answer to whether Codex or Antigravity can build the non-frontend portions.

Product decision summary

Decision	Recommended direction
Name	Undecided; use a neutral repository/package identifier for now.
Core platform	Windows-first desktop companion; keep boundaries portable for later macOS/Linux support.
Storage	Local-first encrypted vault; no central readable-notes database; optional user-controlled encrypted sync later.
Desktop UX	Native lightweight tray + quick-capture window; dashboard loads on demand in a webview/browser.
Frontend	React + TypeScript + Vite; PWA/dashboard; extension UI in TypeScript.
Backend/core	Go service, SQLite, local attachment store, local API and MCP server.
AI integration	MCP is optional and scoped; AI never required for core note capture or search.
Part 2	Watchlist for anime/films and preference-aware recommendations; mobile app considered later.

1. Product vision and feature scope

Promise: capture quickly, keep control of the data, and find it later. The product should feel like a friendly desktop companion, not a heavyweight productivity suite.

Core features

- Quick capture via global shortcut; system tray; optional floating sticky notes; dashboard/PWA.
- Note types: thought, idea, task, reminder, link, image, quote/snippet and reference.
- Markdown, checklists, code snippets, multiple image attachments, source URL/title and timestamps.

- Inbox-first workflow; collections/spaces; tags; pin, archive, restore, snooze and bulk actions.
- Full-text search with SQLite FTS5; filters by date/type/tag/collection/status.
- Browser extension: save page, selected text, image, screenshot and quick note; preserve source metadata.
- Optional pet companion with idle, sleep, saved-note, reminder and celebration states; toggle/hide/reduced-motion controls.
- Memory lane to resurface older notes; user-configurable frequency.
- Export/import: JSON + Markdown + attachments in ZIP; documented portable format.

2. Privacy and storage model

Interpretation: the product operator should not have access to readable note content. Data still needs a home, so the recommended default is on-device storage, encrypted at rest. No mandatory account or hosted note database.

- SQLite stores structured records and search index; encrypted attachment files live in an app-managed directory.
- Vault key protected using OS credential storage where practical; use established cryptographic libraries and authenticated encryption, never custom crypto.
- Local-only mode works offline. Optional sync later should be user-controlled (WebDAV/S3-compatible destination) or end-to-end encrypted; server should see ciphertext only.
- No content in analytics, logs, crash reports or telemetry. Diagnostics opt-in and redacted.
- MCP and external AI access are separate trust boundaries: only selected fields/collections leave the device after user consent.
- Threats include malware on the user's account, stolen devices, malicious MCP clients, extension compromise, weak backups and accidental exports.

Privacy promise must be precise

Local encrypted storage can prevent the operator from reading stored notes. It cannot protect against malware or a compromised user account. If the user asks an external AI provider to process note text, that selected text is sent to that provider and governed by its policies. Make this explicit in the UI.

3. Entire application architecture

Recommended component topology

Layer	Component	Role / boundary
User surfaces	Desktop tray + quick capture; web dashboard/PWA; browser extension; later mobile app	Capture and display only. Do not each implement their own storage logic.
Application core	Go application service	Single source of business rules: notes, collections, attachments, vault, search, watchlist, permissions.
Adapters	Local HTTP API / Native Messaging / MCP	Translate requests into core service calls; authenticate and enforce scopes.
Persistence	SQLite + encrypted file store	Local user-owned vault. Database migrations, FTS index, attachment references.
Optional external	User-selected sync destination / metadata API / AI client	Only encrypted sync payloads or user-approved content. Not required for core operation.

Architecture map (logical flow)

```
[Desktop tray / Quick Capture] ■■
[Web dashboard / PWA] ■■■■■■■■■■■■> [Go Application Service]
[Browser Extension] ■■■■■■■■■■■■ ■
[MCP Client: Claude/ChatGPT*] ■ ■■■> [Vault / Permission Checks]
■ ■■■> [SQLite + FTS5]
■ ■■■> [Encrypted Attachments]
■■■> [Reminder / Export / Import]

Optional later:
```

```
[Go Sync Adapter] <■■ ciphertext only ■■> [User's own storage destination]

*Only where the user's AI client supports the relevant MCP connection mode.
```

Key rule: all surfaces call the same Go application service. The extension and MCP server must not access SQLite directly or bypass validation. The service layer is the policy boundary.

3.1 Core runtime and request lifecycle

- App starts → initialise config → unlock vault (or remain locked) → open SQLite → run migrations → start local API → register tray and shortcut.
- A user action arrives from a surface → adapter authenticates caller → checks permission/scope → validates input → calls application service.
- Service applies business rules → repository writes to SQLite / attachment store → transaction commits → event/result returned to caller.
- UI updates from response or lightweight event notification; no polling loop required.
- When locked/quit → stop accepting requests → flush/close DB → clear in-memory key material where practical.

3.2 Capture workflow

```
Hotkey → small capture window → enter text / paste image
→ choose optional type or collection → Save
→ Go service validates → encrypts/persists → returns note ID
→ popup closes / shows confirmation → dashboard refreshes on next open
```

3.3 Browser extension workflow

```
User clicks extension / context-menu action
→ choose page / selected text / image / screenshot
→ extension builds capture payload + source URL/title
→ authenticated local bridge (or Native Messaging)
→ Go service checks origin, token and payload limits
→ save note + attachment → return success/failure
→ extension shows concise confirmation
```

- Request minimum browser permissions; avoid persistent access to all websites.
- Protect localhost bridge against drive-by website requests (random token, strict origin checks, host validation, loopback binding, request-size limits).
- Native Messaging offers a tighter extension-host boundary but more packaging friction; prototype both and choose based on security/usability.

3.4 MCP request workflow

```
User asks AI assistant to find/save something
→ AI client invokes MCP tool
→ local MCP adapter authenticates client + checks granted scope
→ Go application service runs search/create/update operation
→ only permitted result fields returned to AI client
→ user sees action/result; write operations are logged without note body
```

- Start read-only by default. Add create/update only after explicit per-client write permission.
- Scopes: metadata read, note-content read, attachment read, create, edit, archive, permanent delete, watchlist read/write.
- Permanent delete should be separate from archive and require explicit confirmation.
- Revoke client access and rotate credentials from settings. Bind local-only by default.

3.5 Backup, restore and sync workflow

- Backup creates an encrypted archive with manifest, schema version and checksums.
- Restore validates archive, prompts for vault credentials and restores into a new vault before replacing current data.

- Sync is a later feature: encrypt locally before upload; use versioning and conflict resolution; do not upload plaintext or keys.
- Do not promise secure deletion from SSDs by overwriting files; explain trash and backup retention.

4. Tech stack and developer tools

Area	Recommended choice	Why / alternatives
Language/core	Go	Simple deployment, concurrency, fast startup potential. Rust is viable but likely slower to iterate for this project.
Desktop tray	Go systray library + platform shortcut/window adapter	Prototype library maturity on Windows. Native Windows APIs may be needed for robust shortcuts and popup behaviour.
Desktop window	Small native window or lightweight WebView opened on demand	Avoid resident Electron. Wails is an option if integrated UI matters more than smallest idle footprint.
Dashboard	React + TypeScript + Vite	Simple SPA/PWA; Next.js is unnecessary unless server-rendering or public pages are needed.
Styling/motion	CSS variables + Tailwind (optional) + Motion/Framer Motion	Use CSS transitions for most interactions; motion library only where useful.
Database	SQLite + FTS5; migrations via a Go migration library	Single-user local-first; avoid running a database server.
Crypto	Go standard crypto + vetted maintained libraries; OS keychain integration	Use Argon2id key derivation and authenticated encryption through reputable implementations; security review required.
API	Go net/http or Chi; JSON over loopback	Small local API; OpenAPI spec for typed clients if useful.
MCP	Official Go MCP SDK / maintained MCP SDK	Keep MCP adapter separate from domain logic; verify SDK support and transport at implementation time.
Extension	TypeScript + WebExtensions API; Chrome MV3 first	Firefox later; Native Messaging vs authenticated localhost bridge spike.
Testing	Go testing, testcontainers only if needed, Playwright, extension manual test matrix	Unit tests for domain rules; integration tests for DB/vault; end-to-end tests for capture.
Quality	golangci-lint, gofmt, govulncheck, ESLint, Prettier, TypeScript strict	CI on every pull request; dependency and secret scanning.
Build/release	GitHub Actions, signed Windows installer, release artifacts	Start with manual releases; auto-update only after reliable signing/update design.
Design	Figma or Penpot; design tokens in CSS	Define components, theme tokens, motion rules and responsive breakpoints before UI expansion.

Tooling setup

- Editor: VS Code or Cursor; Go extension, ESLint, Prettier, GitLens optional.
- AI coding agent: Codex or Antigravity as an implementation partner; keep architecture notes and task specs in the repository.
- Version control: Git + GitHub; small branches, review diffs, protect main.
- API debugging: curl/HTTPie and Bruno/Postman; SQLite Browser for local development only (never inspect real user vaults without consent).
- UI design: Figma/Penpot; icon set from Lucide; assets as optimised SVG/sprite sheets.
- Observability: local structured logs with redaction; opt-in crash reports; no third-party analytics containing content.

5. Codex or Antigravity: enough for everything except frontend?

Short answer: yes, either can assist with most non-frontend engineering—Go core, SQLite repositories, API, MCP server, tests, documentation, CI and build scripts. But neither should be treated as an autonomous architect or security auditor. The hard parts are native desktop integration, encryption/key lifecycle, extension

permissions, MCP security and cross-platform packaging.

Suggested division of work

Workstream	AI coding agent can do	You must verify
Go core / services	Scaffold packages, implement CRUD, repositories, validation, migrations, tests.	Domain boundaries, error handling, transaction safety, race conditions, API compatibility.
SQLite / search	Schema, migrations, FTS5 queries, repository tests.	Migration safety, query correctness, backup/restore, encryption interaction.
Local API	Endpoints, auth middleware, OpenAPI, tests.	Loopback binding, origin/host checks, token lifecycle, CSRF-like request abuse.
MCP server	Tool/resource definitions, schema validation, client examples.	Scopes, data minimisation, client revocation, prompt-injection boundaries.
Desktop native layer	Tray integration, shortcut prototype, installer scripts and platform wrappers.	Actual Windows behaviour, focus/window lifecycle, startup, sleep/resume and packaging.
Extension bridge	Message schemas, extension service worker, native messaging host or localhost client.	Permission model, browser review requirements, localhost attack surface.
Security	Threat-model draft, tests, dependency scanning, crypto integration scaffolding.	Independent review of cryptographic design and threat model; never trust generated crypto.
CI / release	GitHub Actions, lint/test workflows, build matrix, release notes.	Signing, installer validation, upgrade/uninstall and recovery tests.

How to use the agent effectively

- Give it one bounded issue at a time, with acceptance criteria and files/packages it may change.
- Ask for a plan before code on architectural tasks; review the plan, then approve implementation.
- Require tests with every feature and ask the agent to explain failure cases and security assumptions.
- Keep a human-reviewed architecture decision record (ADR) for database, transport, encryption and sync.
- Never paste real private notes, vault keys, tokens or personal data into coding-agent prompts.
- Run tests locally; inspect diffs; do not merge generated code just because the agent says it is complete.

Practical recommendation

Pick one primary agent rather than running both on the same branch. Use Codex if you want a repository-centric coding workflow and direct implementation/testing in the codebase; use Antigravity if its agent-led IDE workflow suits how you prefer to supervise multi-file changes. You can use either for backend and infrastructure; frontend can remain your own work. A second agent can be used for independent review, not competing edits.

6. MCP feature catalogue

Tool	Purpose	Permission
create_note / save_link	Create note or save URL and source metadata.	Write
search_notes / get_note	Search and retrieve selected note content.	Read content
update_note / archive_note	Edit or archive a note.	Write
list_collections / create_collection	Browse or create collections.	Metadata read / write
get_due_items	Find snoozed tasks and reminders.	Read
get_watchlist / get_taste_profile	Read watch history and explicit preferences.	Watchlist read
add_to_watchlist / update_progress / rate_title	Track a title and feedback.	Watchlist write
prepare_context_pack	Build a selected bundle of notes/links for coding or research assistant.	Explicit selected-content read

Additional MCP use cases

- Personal knowledge librarian; related-idea discovery; duplicate detection and suggested tagging.
- Learning companion: create flashcards, quizzes and revision summaries from selected notes.
- Project context pack: expose only selected notes/links to a coding assistant.
- Reading queue: summarise user-selected saved articles and store takeaways.
- Task triage: propose task/idea/reference classifications; user approves changes.
- Watchlist taste assistant: recommend titles based on genres, ratings, exclusions and feedback.
- Weekly personal review: summarise notes/tasks saved during a chosen time window.
- All tools remain permissioned; prompts do not grant access.

7. Part 2: Watchlist and taste profile

- Track anime, films and series; optionally manga/manhwa/manhua later.
- Statuses: planned, watching, on hold, completed, dropped, rewatching; progress by season/episode.
- Genres, tags, rating, favourite, short review, dates and private notes.
- Separate watched genres from liked genres; viewing a title does not prove a user likes all its genres.
- Explicit preferences: favourite/disliked genres, tone, pacing, length, romance/comedy balance and content boundaries.
- Recommendation feedback: interested, not interested, already watched, not my taste.
- Start with explainable rules; later use local embeddings for similarity. Use LLMs to explain, not invent catalogue facts.
- MCP can read the user's profile and history only after permission; write a watchlist update only when explicitly requested.
- Mobile companion can be evaluated after desktop v1; share data model and encrypted sync rather than duplicating databases.

8. Visual identity and motion

Token	Hex	Purpose
Ink	#20212B	Text and bold outlines
Canvas	#FFFDF8	Warm background
Violet	#7057D9	Primary actions
Mint	#DDF7EA	Positive/complete
Coral	#FF8D78	Accent/highlights
Butter	#FFE58F	Pinned notes
Blue	#DCEBFF	Informational cards

- Themes: neobrutalist default, coder/terminal, cutesy pastel, anime-inspired editorial, focus/minimal.
- Themes change typography, border weight, shadows, backgrounds, accents, sounds (off by default), animation and companion sprite.
- Use CSS variables/design tokens; avoid hardcoding theme values in components.
- Motion: 120–220 ms micro-interactions, 180–300 ms panels, 250–450 ms pet reactions; tune by feel and device.
- Prefer transform/opacity; avoid continuous ambient animation, heavy blur and large animated shadows.
- Respect reduced motion; pet can be hidden; keep keyboard and screen-reader access.

9. Phase-wise roadmap

Phase	Timeline	Deliverables / exit gate
0. Spikes	Week 1	Windows tray/shortcut proof, Go+SQLite proof, crypto/key approach prototype, architecture decisions.
1. Private capture MVP	Weeks 2–4	Go core, tray, quick capture, local vault, CRUD, collections/tags, images, search, lock/unlock, export/restore.
2. Dashboard & delight	Weeks 5–6	React dashboard/PWA shell, inbox/library, themes, sticky cards, pet, reduced motion.
3. Browser extension	Weeks 7–8	Chrome extension saves page, selection, image and note; secure bridge and permissions.
4. MCP v1	Weeks 9–10	Local MCP read/search/create/update/archive, resources, scopes, revoke and audit.
5. Harden & release	Weeks 11–12	Installer, backup/restore, security pass, accessibility, performance, docs and privacy notice.
6. Watchlist Part 2	Weeks 13–16	Media tracking, genres, ratings, taste profile, feedback and MCP recommendations.
7. Mobile & sync	Weeks 17–22+	Evaluate mobile app, user-controlled encrypted sync and conflict handling.

Assumption: one developer working part-time. Treat estimates as planning ranges; native integration, encryption review and packaging may add time. Keep a demoable build at each phase.

10. Data model and repository layout

Suggested tables

Table	Important fields
notes	id, title, body, type, status, collection_id, pinned, created_at, updated_at, archived_at, snoozed_until
collections	id, name, colour, icon, sort_order, created_at
tags / note_tags	tags: id, name; join: note_id, tag_id
attachments	id, note_id, relative_path, mime_type, size, checksum, created_at
watch_items	id, title, media_type, status, progress, rating, year, external_ref
watch_genres / preferences	title genres; explicit likes/dislikes; inferred preferences with confidence/source
mcp_clients / permissions	client_id, scopes, token_hash, created_at, last_used_at, revoked_at
audit_events	id, client_id, action, target_id, timestamp, outcome; no note body

Suggested repository layout

```
/cmd/desktop/ Go desktop app entry
/cmd/mcp/ MCP server entry (or desktop subcommand)
/internal/app/ business use cases and domain services
/internal/store/ SQLite repositories and migrations
/internal/vault/ encryption, lock/unlock, key lifecycle
/internal/desktop/ tray, shortcuts and native windows
/internal/http/ local API and auth middleware
/internal/mcp/ MCP tools, resources, prompts, scopes
/internal/sync/ future sync adapters
/web/ React dashboard/PWA
/extension/ TypeScript browser extension
/assets/ icons, themes, sprites and sounds
/docs/ ADRs, threat model, export format
```

11. Risks and scope boundaries

- Do not build hosted sync, mobile, social features, marketplace, OCR, voice or AI auto-organisation in the MVP.
- Native desktop integration is platform-specific: Windows-first and prove it early.
- Encryption is high stakes: use vetted libraries and get independent review before strong security claims.

- MCP is an attack surface: least privilege, local auth, revocation and data minimisation.
- Avoid feature creep: watchlist is a separate Part 2; mobile app follows only after data model and sync design are stable.
- Catalogue metadata must come from permitted sources with terms/attribution respected.

12. MVP acceptance checklist

- Clean Windows install; tray and configurable global shortcut work.
- Create/edit/pin/archive/restore/delete notes; collections and tags work.
- Multiple image attachments save and display; search and filters work.
- Vault locks/unlocks; encryption at rest is verified; backup restores on a fresh profile.
- Offline core use; no mandatory account or central readable-notes storage.
- Browser extension captures selected page/text safely (Phase 3).
- MCP scopes are independently configurable and revocable (Phase 4).
- Theme switch coherent; pet optional; reduced-motion and keyboard navigation work.
- No note content in logs/telemetry; external AI processing is explicit.

Recommended first milestone

Build one perfect loop: press shortcut → jot a thought or paste an image → save locally → close → search and find it later. Once that is fast, private and reliable, layer on the visual personality, browser capture and MCP. That sequence reduces risk while preserving the product's distinctive vision.

Expanded working document • Product name intentionally undecided • Timeline is an estimate, not a guarantee.